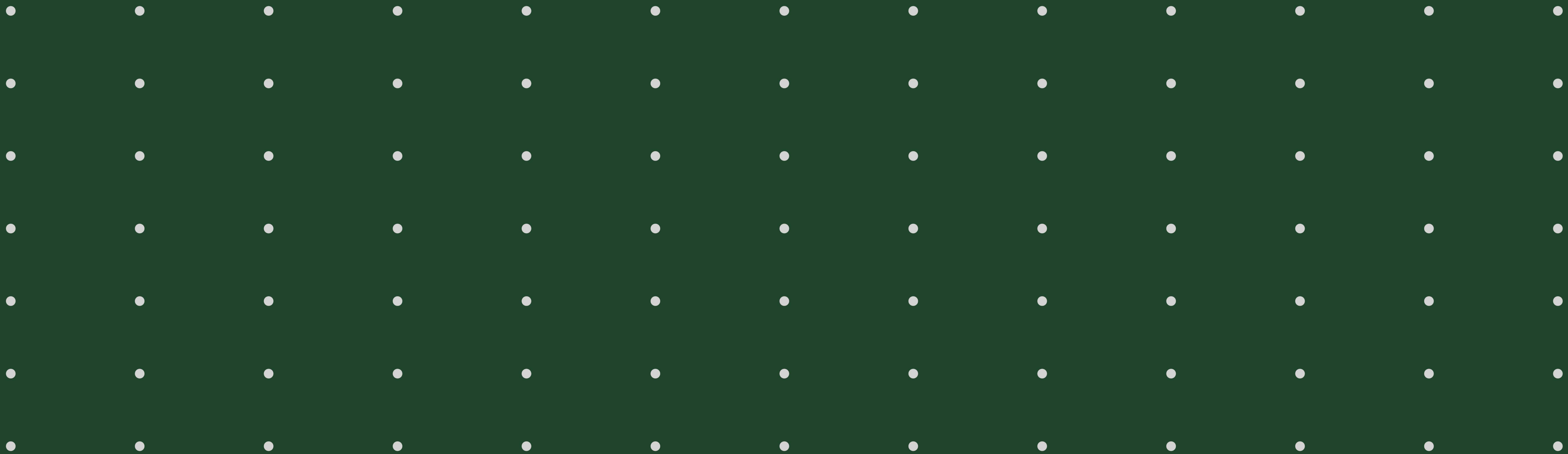


Price Assurance

UC4



Legend

Color key for object states — documentation reference, not a proposal

Green Documented & stable	Yellow Undefined attrs / backend-only	Orange Open questions on record	Gray Named, not modeled	Purple Out of scope / pending elsewhere	Red Documented inconsistency
--	--	--	--------------------------------------	--	---

Price Assurance — Object Model

Source: the object-model doc (“OOUX,” v0.16) and the running open-questions log (“QUESTIONS,” v2.13)

This is a documentation reference, not a design proposal. Object shapes, relationships, and open questions reflect what is currently written in those two docs — nothing here resolves an open item or suggests a direction for an unresolved architecture question.

Read left-to-right: Core PA Objects define the domain. Exception and Dispute clusters branch off Invoice/InvoiceLine. Backend engine sits behind. Shared/external systems and parking lot are separate from the PA-owned model. Open questions are tracked at the end.

How to read this

This tracks the Price Assurance data model across design sessions, Stanford calls, and architecture discussions. It might not be exhaustive — it's meant to give the team a shared picture of what's confirmed, what's still fuzzy, and what questions are still open.

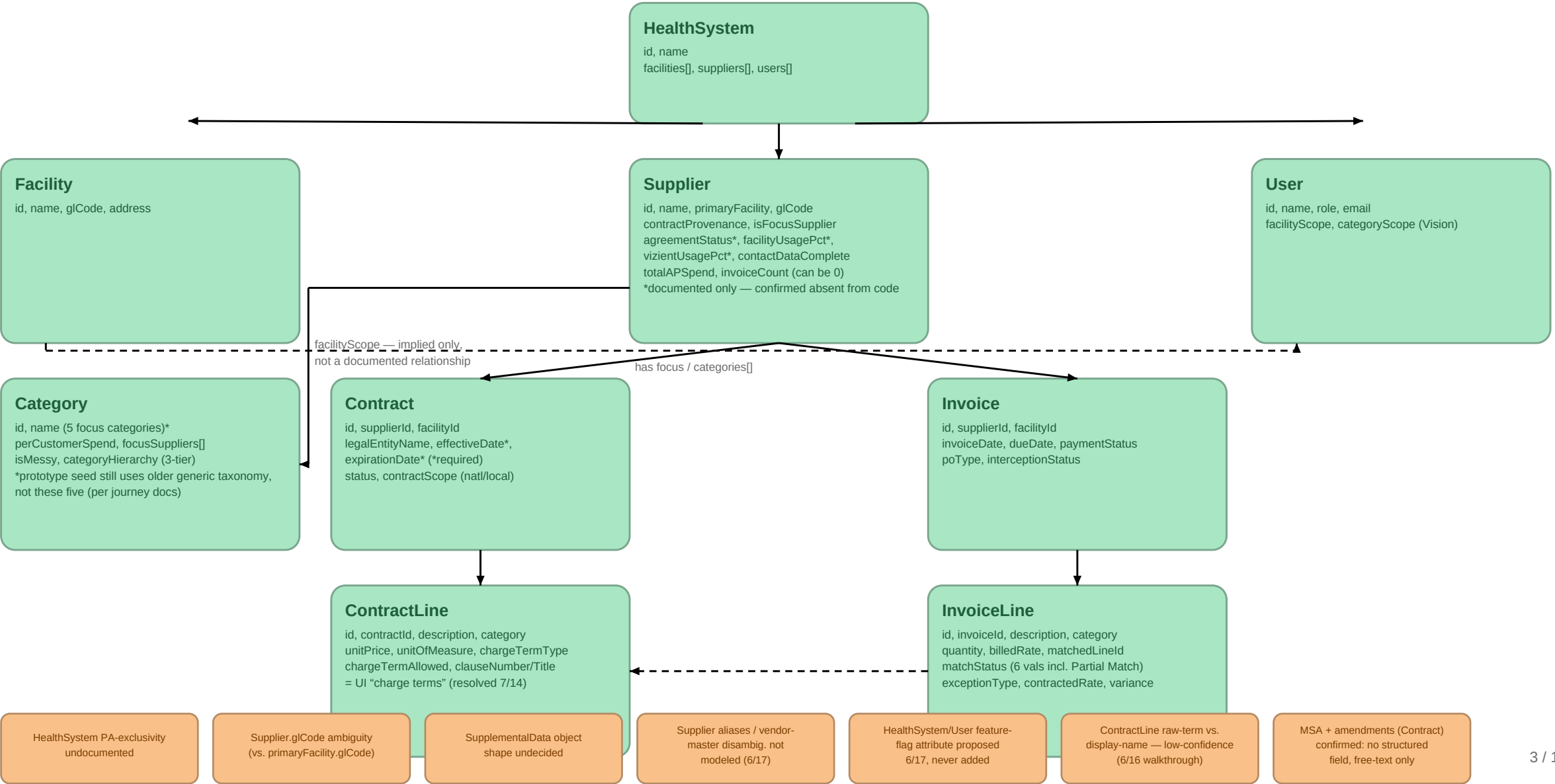
The color coding shows what's safe to build against (green), what exists as an idea but has no shape yet (yellow), what's genuinely undecided (orange), and what's been mentioned but not modeled (gray).

This isn't a spec and isn't final — just a snapshot meant to help avoid building on an assumption that was never actually confirmed, or re-deciding something that's already settled.

If anything here looks wrong or out of date, that's genuinely useful to know — happy to correct the source doc, not just this board.

Core PA Objects

Documented & stable — green cards, attributes as written in OOUX v0.16



Exception Cluster

Detected on InvoiceLine, may escalate to Dispute

Exception

id, invoiceLineId, type, status
detectedAt, varianceAmount, annualizedExposure
explanation, remediationSuggestion
dismissalReason,
humanDecision ("snooze" | "investigate")*
*contradiction: journey docs state prototype has dismiss only, no snooze built yet

AnnualizedExposure

invoiceLineId, varianceAmount
invoiceFrequency (data source open)
projectedAnnualOverpayment

computed, not stored

InvoiceLine (ref)

Dispute (ref)

Exception Types (8, as documented)

Rate Above Contract

Unmapped Charge

Unmatched Line

Expedite Fee

Insufficient Data

Frequency Violation

Unauthorized Line Item

Tax Charge on Tax-Exempt Entity

Second classification
axis (hard-disallowed vs.
misuse-of-allowed)

Source-document
traceability gap

"Rejected"-status-
object-unclear

Dismiss-rollup to
category managers

Invisible-leakage gap —
PO may not be cut at
contracted rate

\$4.75 (Lindy/Linde) vs.
\$4.50 (unnamed Stanford
contract) — not merged

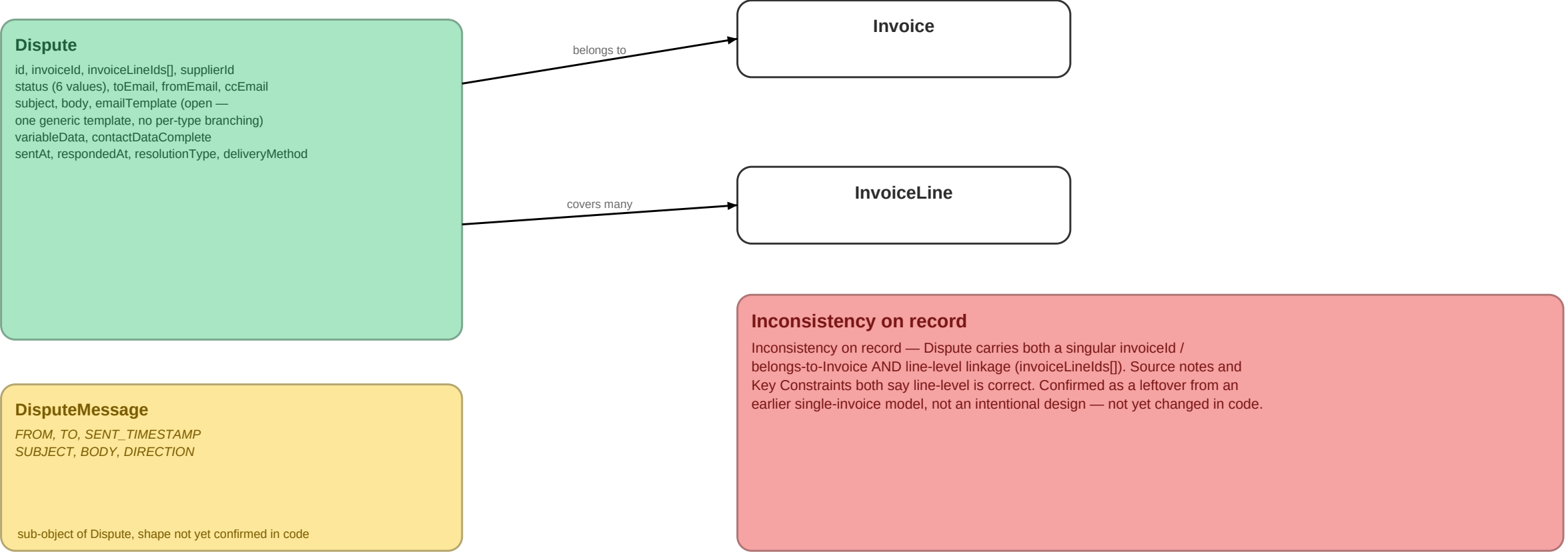
matchStatus ≠ Exception.type — two separate fields

InvoiceLine.matchStatus (6 values, documentation vs. code corrected): Matched, Unmatched, Partial Match, Rejected, No Match, Exception — whether/how confidently a line matched a ContractLine. The engine's separate match outcomes (4): MATCHED, NEEDS_REVIEW, NO_MATCH, REJECT feed into this field. None of these describe what's wrong with the price — that's Exception.type (8 values, left), a different field entirely, generated only after a line is flagged.

Murkiness on record (open): "Rejected" appears as both a matchStatus value AND one of Invoice.paymentStatus's 5 documented values (unexercised in seed data). Whether the backend/ontology model's "Rejected" concept refers to the match-level or payment-level value — or something else — is unresolved; not reconstructed here.

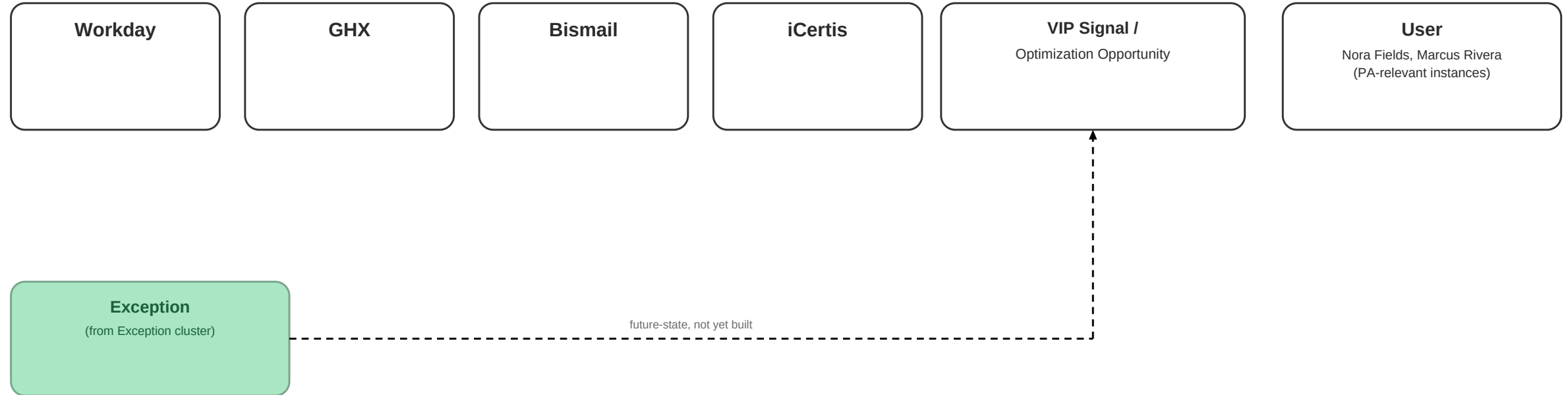
Dispute Cluster

Formal challenge — documented linkage is at the InvoiceLine level



Shared / External

Systems and actors outside the core PA-owned model



GHX: documented as upstream of Workday's 3-way match — an open AI-placement question concerns this system.

Bismail: Vision-scope dispute delivery. iCertis: proposed data source, feasibility unconfirmed.

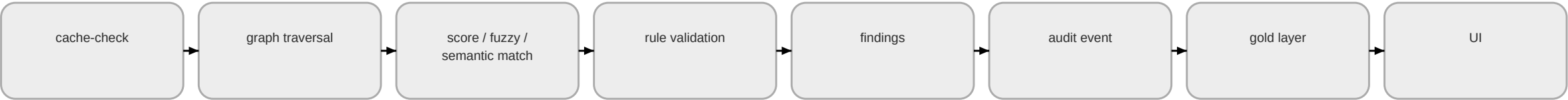
Backend / Engine

ChargeTermOntology pipeline + engine-only objects (yellow = confirmed-but-unshaped)

ChargeTermOntology

backend/conceptual, not a coded type
Types: SUBSCRIPTION, VARIABLE_RATE, OVERAGE,
TIME_AND_MATERIALS, EXCLUDED

documented flow: invoice line → supplier → member → contract → contract version → contract line terms → (above)

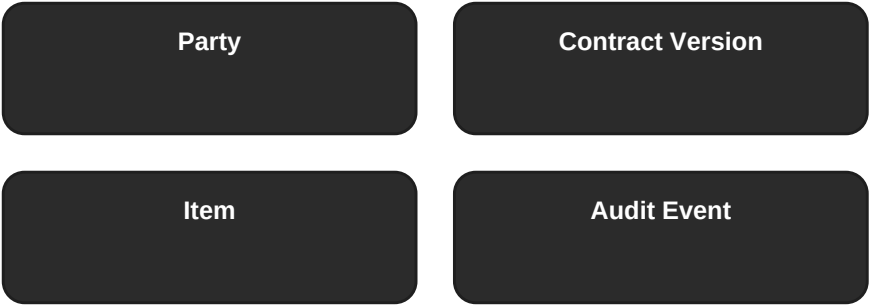


GHX ↔ Workday placement

checked against 7/13 + 7/14 sources — open, unresolved

DetectionRule

6 rules in code, absent from OOUX narrative



candidate edges (named, not modeled):
“governs”, “eligible for”, “has version”

Documented finding: REJECT is dominated by parse failure, not billing violations

Matt Clark engine demo (UHS-Experian data): fast path matched 0 of 31 real invoice lines; fallback PDF extraction produces Qty=1 for every line, generating false REJECT outcomes from bad parsing — not actual overcharges. The prototype does not currently give REJECT distinct UX treatment from the other 3 outcomes, and does not distinguish a parse-failure REJECT from a genuine-violation REJECT — both render identically today. UX treatment still open.

Key Constraints

OOUX v0.16 — verbatim, applies across multiple objects (not one cluster's concern)

No item master — purchased services have no SKUs. All matching is semantic.

No standard UOM — services may have no unit of measure. Must not produce false exceptions when UOM absent.

Two-level matching — engine first selects the correct contract for the supplier (one supplier may have multiple active contracts), then selects the correct charge term within that contract.

Contract versioning — January invoice matches January contract, not February renewal.

Category at line level — both ContractLine and InvoiceLine carry category. Not at Contract or Invoice level.

Dispute at line level — Dispute links to InvoiceLines, not Invoice. Multiple lines from same supplier can be combined.

Contact data required — `supplier.contactEmail` must be populated before dispute initiation.

Also on record but not detailed on this board — some already resolved or covered elsewhere — remaining open items with literal text on the next page.

Additional Open Items

QUESTIONS v2.13 — literal text, object-shape/attribute/relationship items not detailed elsewhere on this board

What is the data source for invoice frequency (monthly/quarterly/annual)?

REJECT vs NO_MATCH — should these surface differently in the UI? Confirmed: today they render identically, no branching in code.

NEEDS_REVIEW / requiresHumanReview — distinct work queue or label only? Confirmed: neither exists in code today — zero occurrences found.

Contract version selection — date-based matching UX?

Invoice parsing quality — how does the UI communicate when a REJECT is caused by parse failure vs billing violation?

Invoice-level status rollup — what is the logic?

How are MSA + amendments modeled? Confirmed: not at all — no versioning field exists; only appears in free-text notes.

How is a corrected invoice modeled?

Payment status model — binary or five-state? Of the 5 documented values, only 2 (Paid, Pending Approval) appear in seed data.

Many-contract-lines-to-one-invoice-line compound matching — what's the architecture? Confirmed: strictly one-to-one today, no compound-match logic.

Invoice versioning at line level?

Can a supplier span multiple facilities? Confirmed: no — Supplier.facility is a singular scalar today, not an array.

No-contract suppliers — flag an exception on every invoice?

Data-acquisition path — iCertis (contract repo) + Workday access?

Open Questions on Record

QUESTIONS v2.13 — literal question text, not just topic labels

What does the undocumented “Partial Match” status (1 of 6 code values) mean product-wise — and is it Summit-relevant or a leftover?

Should dismissal counts and reasons be tracked and surfaced to category managers over time?

What is the exception explanation and remediation copy for each exception type? (Currently one generic template serves all types.)

Is there a check that a not-to-exceed PO was cut at the correct contracted rate in the first place? (“invisible leakage” upstream of Workday’s PO-match control)

What object shape should supplemental data (e.g. laundry pounds) take? [OOUX note only — no formal QUESTIONS ID]

What is the complete, finalized exception-type taxonomy at Summit? (5 divergent lists currently coexist in code)

Does the solution intercept invoices pre-payment, or only analyze post-payment, given Stanford’s Workday auto-approves via a near-instant 3-way match?

Is a “Rejected” status captured in the backend/ontology data model? (unclear whether this concerns invoice, dispute, or exception status)

Are we in Stanford’s invoice-submission stream (into Workday/GHX), or do we receive invoices from Workday after approval?

Do each of the five focus categories sit at the parent tier, or do some nest at the middle tier? [OOUX note only — a related but different formal question defines the tier structure itself, not this mapping]

Parking Lot

Explicitly out of scope — not building against these

RTV / “Returns Not
Deducted” — candidate
exception type

PO budget
burn-down logic,
deferred

iCertis + Workday
data-acquisition path,
feasibility unconfirmed